Notas de Aula
Prof. Adriano Milanez

Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 18]
[Atualizado em: 23/02/2026]

[VITE – Estilização]

1

VITE – ESTILIZAÇÃO

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Estilização** é o processo de aplicar aparência visual aos componentes da interface, como cores, espaçamentos, fontes, tamanhos e outras propriedades CSS.

- ✓ Quando falamos em **estilização** no React, estamos nos referindo a como aplicar esses estilos nos componentes criados com a biblioteca React, que é a base para construir a interface.

- ✓ As técnicas e abordagens de **estilização** (CSS global, CSS Modules, styled components, Tailwind CSS, etc.) são aplicáveis a projetos React independentemente da ferramenta ou framework (VITE, NEXTJS, etc…) que esteja sendo usado para construir o projeto.

- ✓ A escolha da forma de **estilização** depende da complexidade do projeto, das necessidades da equipe e da manutenção futura da aplicação, não da ferramenta que está rodando o React.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ No ecossistema do React, existem várias abordagens para aplicar estilos, cada uma com seus prós e contras.

- ✓ Aqui estão as principais abordagens:
  1. **CSS Global (arquivos .css ou .scss)** – Estilização tradicional carregada globalmente.
  2. **CSS Modules** – Escopo local para cada componente, evitando conflitos globais.
  3. **Tailwind CSS** – Utilização de classes utilitárias para um design consistente e rápido.
  - ✓ **Styled-components** – com suporte a temas e dinamicidade, foi uma das formas mais utilizadas para estilização, porém, em março de 2025, o styled-componentes entrou em **modo manutenção**.
  - ✓ **Inline Styles (style={{ }})** – Estilização direta no JSX, útil para estilos dinâmicos simples.
  - ✓ **SASS/SCSS** – Extensão do CSS com variáveis, mixins e funções avançadas.
  - ✓ **Stitches / Linaria** – Alternativas modernas ao CSS-in-JS, com melhor performance.
  - ✓ **Vanilla Extract** – CSS-in-TypeScript, permitindo tipagem forte e integração com o TS.
  - ✓ **Theme UI / Chakra UI / Material UI** – Bibliotecas com componentes estilizados e temas integrados.
  - ✓ **PostCSS** – Processador para transformar CSS com plugins, como autoprefixer e nesting.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- CSS Global é uma forma tradicional de aplicar estilos em toda a aplicação. Os estilos definidos afetam todos os elementos que correspondem aos seletores, independentemente de onde estão nos componentes.
- É o arquivo CSS que estamos habituados a trabalhar no dia-a-dia.
- **Onde fica?**
  - Geralmente em um arquivo como index.css ou App.css, importado no main.tsx ou App.tsx.
- **Vantagens**
  - Simples de configurar e entender.
  - Útil para definir reset de estilos, tipografia base, cores globais, etc.
- **Cuidados**
  - Como os estilos afetam toda a aplicação, é fácil ter conflito de nomes de classe.
  - Não é ideal para componentes reutilizáveis com estilos específicos.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Problemas comuns**
  - ✓ **Conflitos de classes**: Se várias páginas definirem estilos globais, pode haver **sobrescrições** indesejadas.
  - ✓ **Dificuldade de manutenção**: Em grandes projetos, fica difícil saber onde cada classe está sendo usada.
  - ✓ **Menos modularidade**: Diferente de CSS Modules ou CSS-in-JS, os estilos globais afetam toda a aplicação.
- ✓ **Boas práticas**
  - ✓ Utilize prefixos de classe para evitar conflitos (.btn-primary, .card-container).
  - ✓ Combine CSS Global com CSS Modules para componentes específicos.
  - ✓ Organize os estilos em arquivos menores para facilitar manutenção.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**
     - ✓ **Útil para:**
       - ✓ Reset de estilos (Normalize.css, Tailwind Preflight).
       - ✓ Estilização de elementos HTML básicos (body, h1, button).
       - ✓ Definição de temas globais (:root { --primary-color: #0070f3; }).
     - ✓ **Evite em:**
       - ✓ Estilização específica de componentes (use CSS Modules ou Styled Components).
       - ✓ Projetos grandes sem organização, pois pode gerar conflitos.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**

     - ✓ **Vamos criar nossa TERCEIRA aplicação VITE**

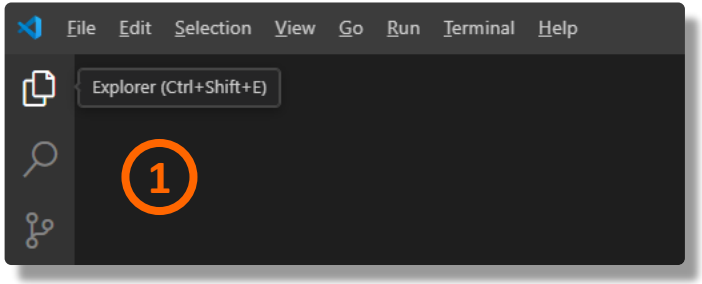
       - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.

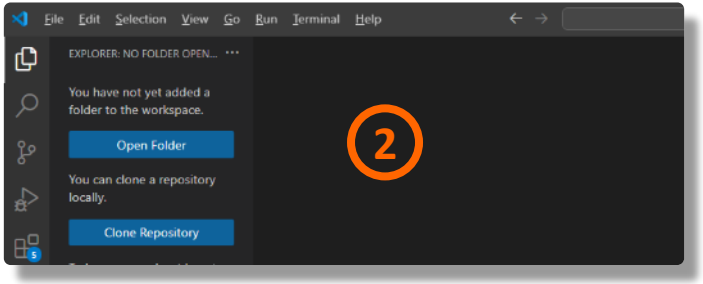
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    1. Selecionamos o ícone ‘Explorer’
    2. Na sequência, clique em ‘Open folder’



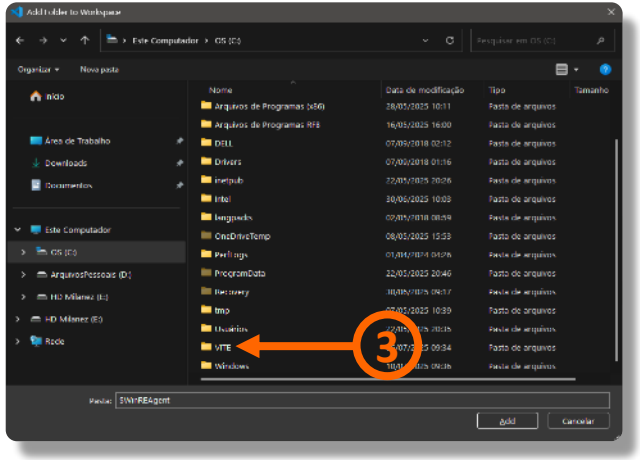


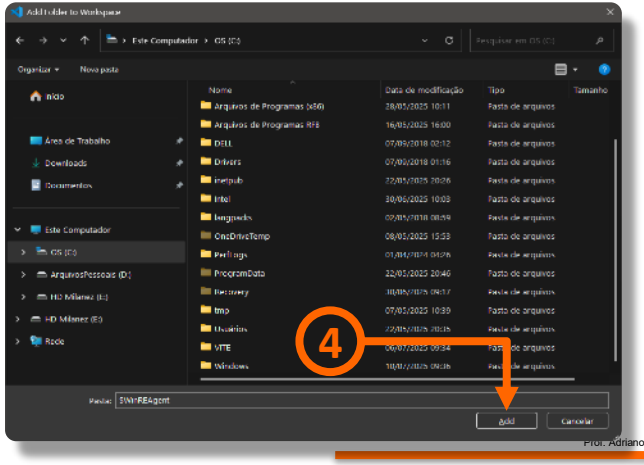
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    3. Procure a pasta ‘VITE’, que criamos anteriormente e selecione.
    4. Na sequência clique em ‘Add’.





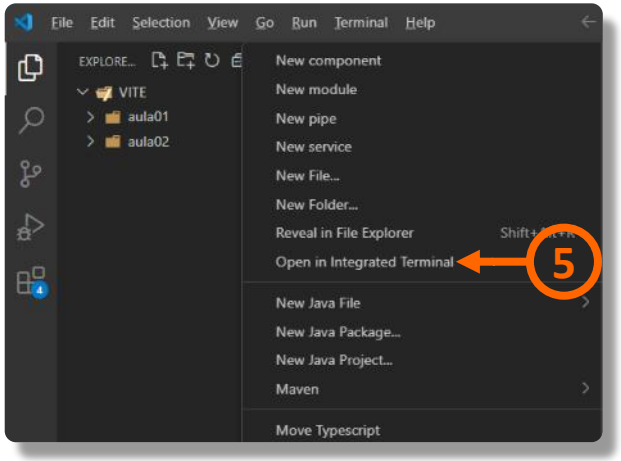
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code

5. Em uma área qualquer sem conteúdo, clicamos com o botão direito do mouse e selecionamos ‘Open in Integrated Terminal’



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

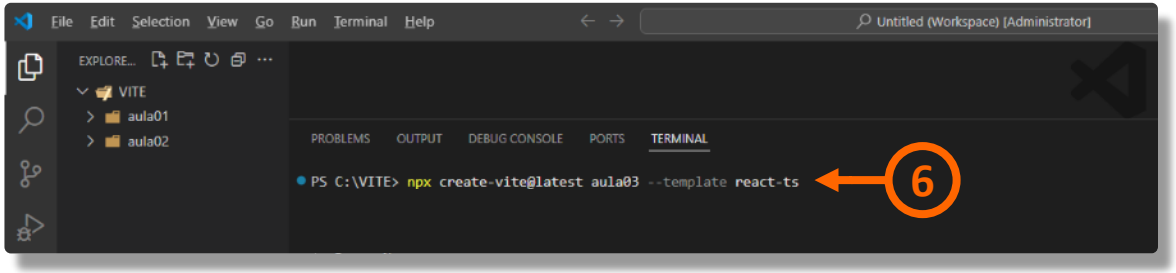
- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code

6. Na sequência digitamos a instrução

   ‘`npx create-vite@latest` **`nome-projeto`** `--template react-ts`’

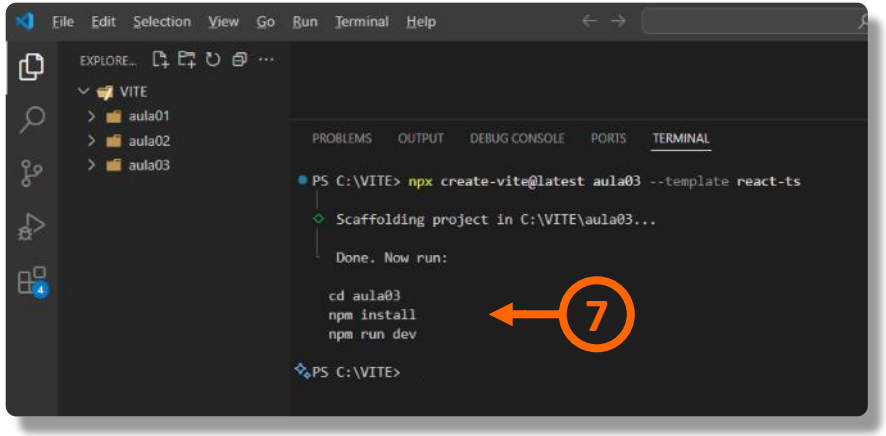


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    7. Após a criação do projeto teremos a seguinte resposta.

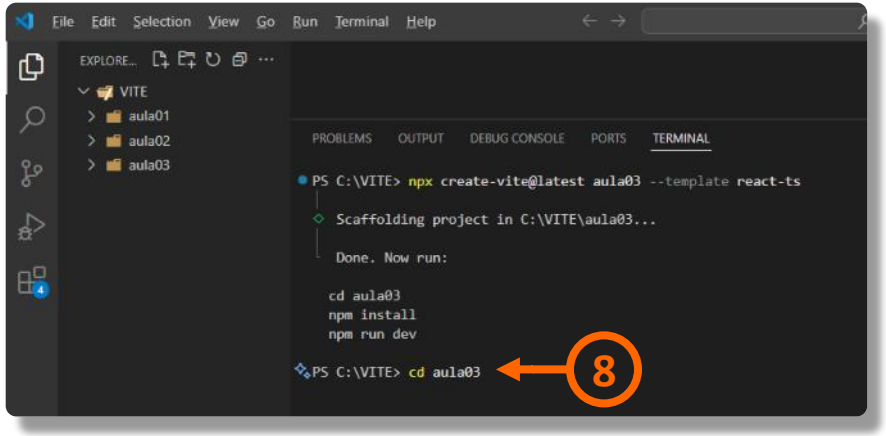


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    8. Como na imagem, vamos fazer o que se pede, no prompt digitamos ‘`cd aula03`’.

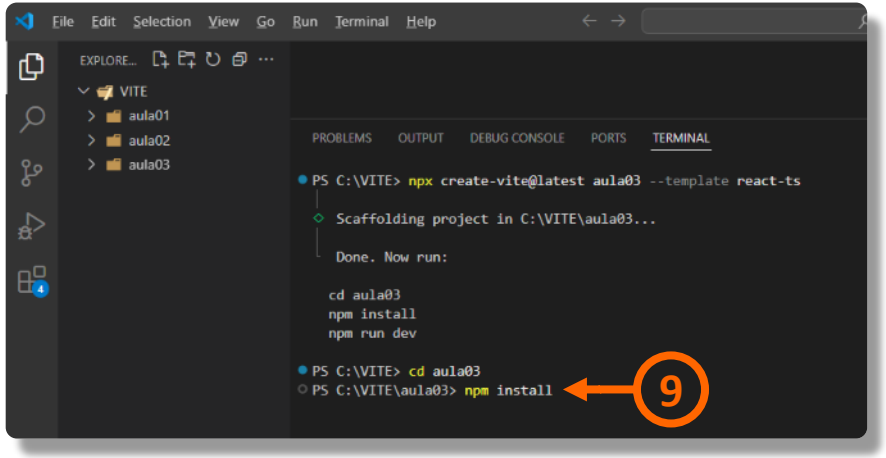


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    - 9. Agora digitamos '`npm install`'.

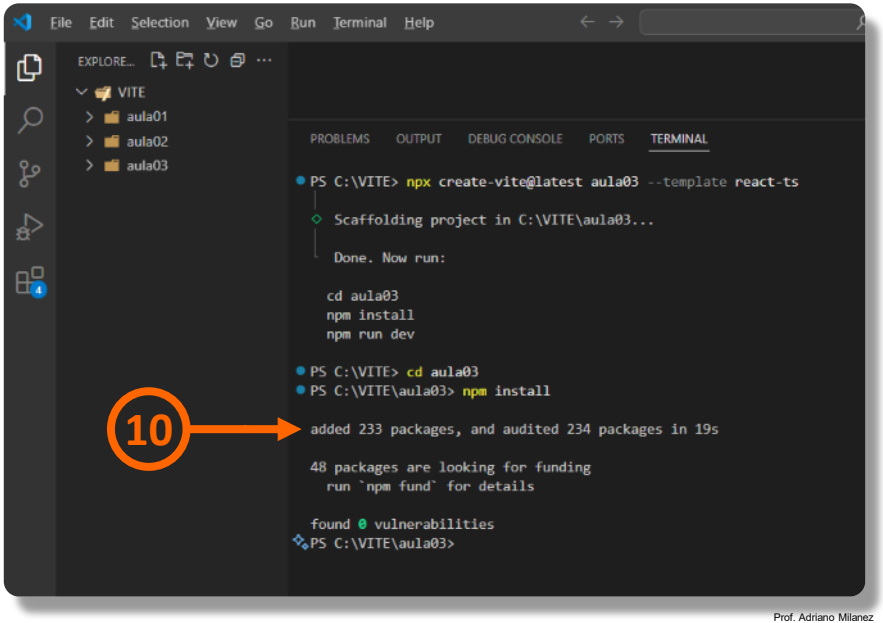


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos criar nossa aplicação de uma forma um pouquinho diferente.
  - ✓ Vamos abrir o VS Code
    - 10. Como resultado temos a instalação dos pacotes necessários para que nossa aplicação 'rode'.

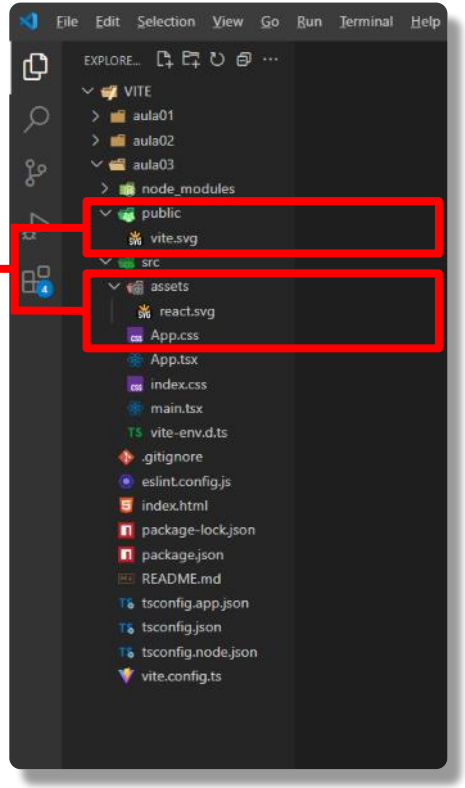


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**

     - ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
       - ✓ Vamos modificar os arquivos criados automaticamente.
       - ✓ Vamos apagar arquivos que não usaremos.
         - ✓ Conteúdo da pasta **PUBLIC**
           - ✓ VITE.SVG
         - ✓ Conteúdo da pasta **ASSETS**
           - ✓ REACT.SVG
         - ✓ Arquivos da pasta **SRC**
           - ✓ APP.CSS



Lembrando que vamos reescrever nossos arquivos MAIN.TSX e modificar o arquivo APP.TSX

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**
     - ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
       - ✓ Vamos modificar o arquivo **INDEX.CSS**

```
1.  body {
2.    margin: 0;
3.    font-family: Arial, sans-serif;
4.    background-color: #f3f4f6;
5.    color: #333;
6.  }
7.  header {
8.    background-color: #038009;
9.    color: white;
10.   padding: 1rem;
11.   text-align: center;
12.   font-size: 1.5rem;
13. }
14. main {
15.   padding: 2rem;
16. }
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos modificar o arquivo **APP.TSX**

```
1.  import './App.css'
2.  const App = () => {
3.    return (
4.      <>
5.        <header>Meu Site Simples</header>
6.        <main>
7.          <p>Bem-vindo à minha primeira aplicação com CSS global!</p>
8.        </main>
9.      </>
10.   )
11. }
12. export default App
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**

     - ✓ **Vamos criar nossa TERCEIRA aplicação VITE**

       - ✓ **OBSERVAÇÃO**

         - ✓ Apesar do arquivo CSS ser um arquivo estático e poder ser armazenado na pasta PUBLIC **NÃO É** uma prática recomendada para a estilização principal da aplicação.
         - ✓ A pasta public/ é usada para arquivos estáticos que:
           - ✓ Não passam por processamento do Vite (como minificação ou importações).
           - ✓ Ficam acessíveis diretamente via URL (ex: /logo.png, /manual.pdf, etc).
           - ✓ São úteis para favicon, imagens fixas, manifest.json, arquivos externos de configuração, etc.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  1. **CSS Global (arquivos .css ou .scss)**

     - ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
       - ✓ **OBSERVAÇÃO**
         - ✓ **Por que não é recomendado usar public/ para o CSS principal?**
           - ✓ Você não pode importar esse CSS diretamente nos arquivos .ts ou .tsx com import './style.css'.
           - ✓ Você teria que incluir manualmente no index.html com <link rel="stylesheet" href="/estilos.css">, o que foge do fluxo do React + Vite.
           - ✓ O CSS não será processado por ferramentas do Vite (como PostCSS, Tailwind, autoprefixer etc.).

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos modificar o arquivo **APP.TSX**

```
1.  const App = () => {
2.    return (
3.      <>
4.        <header>Meu Site Simples</header>
5.        <main>
6.          <p>Bem-vindo à minha primeira aplicação com CSS global!</p>
7.        </main>
8.      </>
9.    )
10. }
11. export default App
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos modificar o arquivo **MAIN.TSX**

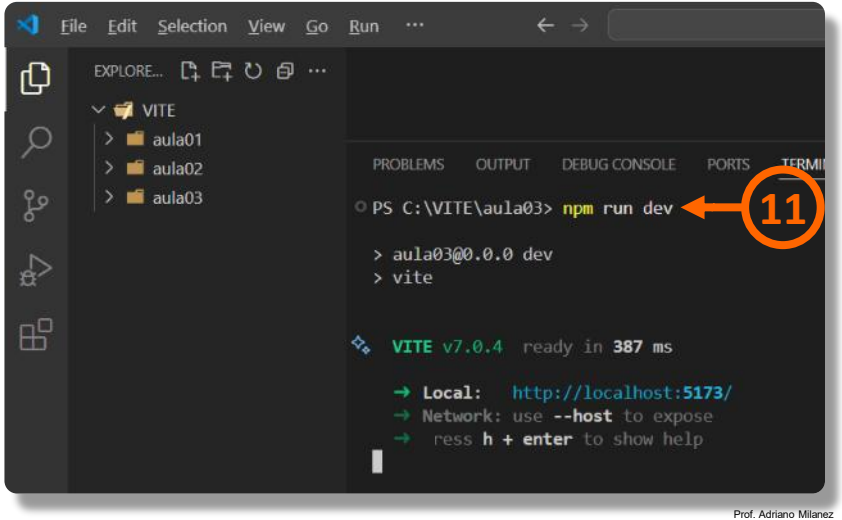
```
1. import { StrictMode } from 'react'
2. import { createRoot } from 'react-dom/client'
3. import './index.css'
4. import App from './App.tsx'
5. createRoot(document.getElementById('root')!).render(
6.   <StrictMode>
7.     <App />
8.   </StrictMode>,
9. )
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    - 11. Vamos digitar no prompt ‘`npm run dev`’ para que consigamos ‘rodar’ nossa aplicação.



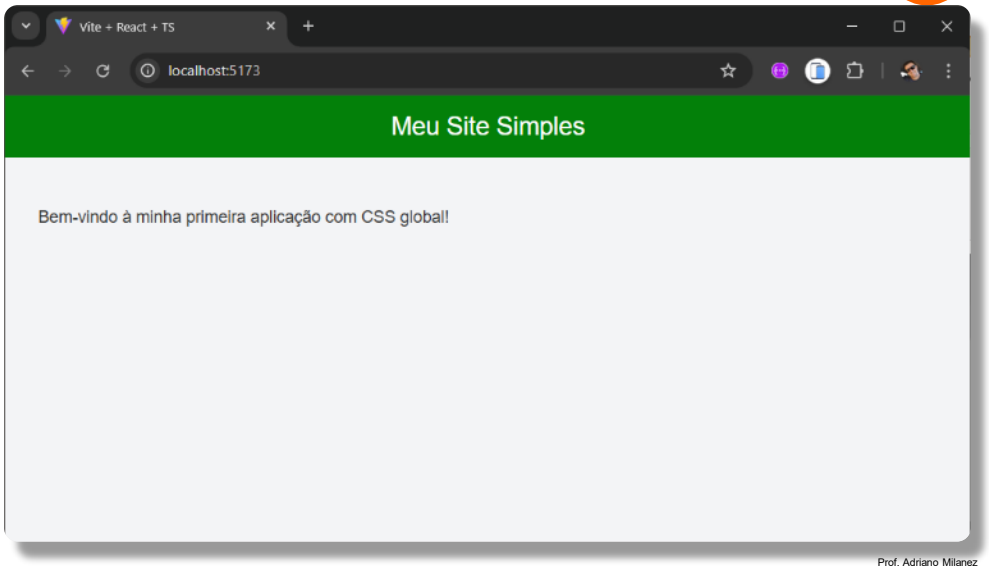
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

1. CSS Global (arquivos .css ou .scss)

- ✓ **Vamos criar nossa TERCEIRA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    - 12. Como resultado temos:

12



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ CSS Modules são uma forma de escrever arquivos CSS onde os nomes das classes são isolados por componente. Isso evita conflitos de nomes e facilita a manutenção dos estilos.
- ✓ **Onde fica?**
  - ✓ São armazenados dentro da pasta src/, geralmente ao lado do componente que eles estilizam.
- ✓ **Vantagens**
  - ✓ Escopo local (as classes só afetam o componente onde são importadas).
  - ✓ Ótimo para projetos com muitos componentes.
  - ✓ Já funciona nativamente com Vite + React + TypeScript (sem configuração extra).

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ CSS Modules são uma forma de escrever arquivos CSS onde os nomes das classes são isolados por componente. Isso evita conflitos de nomes e facilita a manutenção dos estilos.
- ✓ **Boas práticas com CSS Modules**
  - ✓ Use nomes descritivos para classes (.botaoPrimario, .cardContainer).
  - ✓ Mantenha os arquivos próximos aos componentes (/components/Button/button.module.css).
  - ✓ Evite estilos globais, exceto para resets e temas.
  - ✓ Combine com Tailwind CSS ou Styled Components, se necessário.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

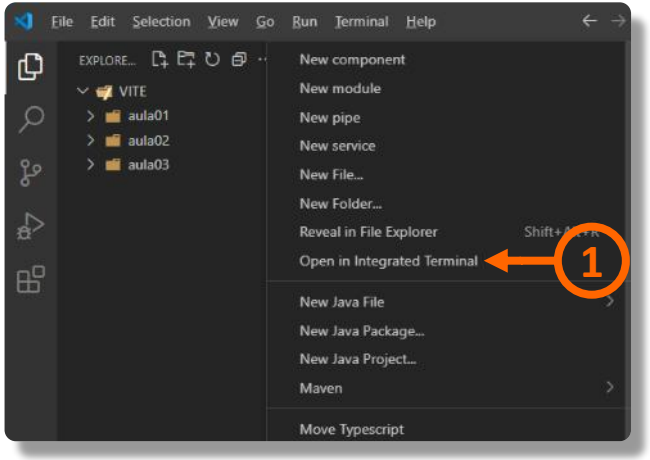
- ✓ CSS Modules são uma forma de escrever arquivos CSS onde os nomes das classes são isolados por componente. Isso evita conflitos de nomes e facilita a manutenção dos estilos.
- ✓ **Boas práticas com CSS Modules**
  - ✓ Use nomes descritivos para classes (.botaoPrimario, .cardContainer).
  - ✓ Mantenha os arquivos próximos aos componentes (/components/Button/button.module.css).
  - ✓ Evite estilos globais, exceto para resets e temas.
  - ✓ Combine com Tailwind CSS ou Styled Components, se necessário.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    1. Em uma área qualquer sem conteúdo, clicamos com o botão direito do mouse e selecionamos ‘Open in Integrated Terminal’



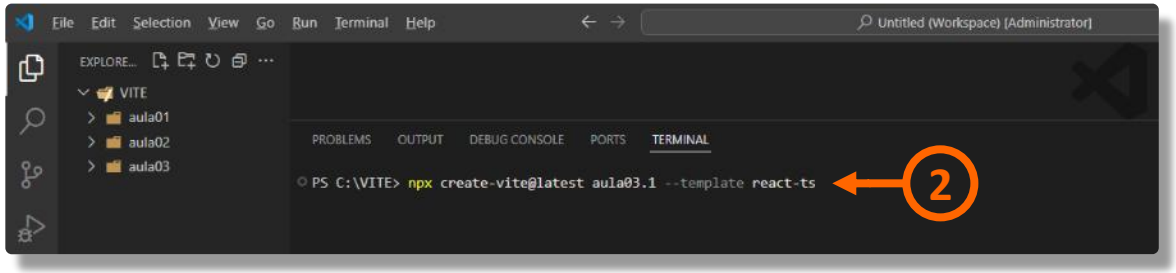
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    2. Na sequência digitamos a instrução

    ‘`npx create-vite@latest` **`nome-projeto`** `--template react-ts`’

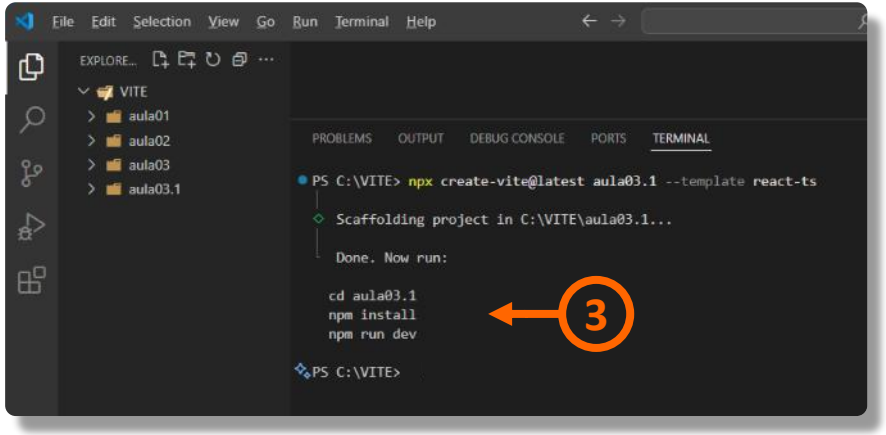


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    - 3. Após a criação do projeto teremos a seguinte resposta.

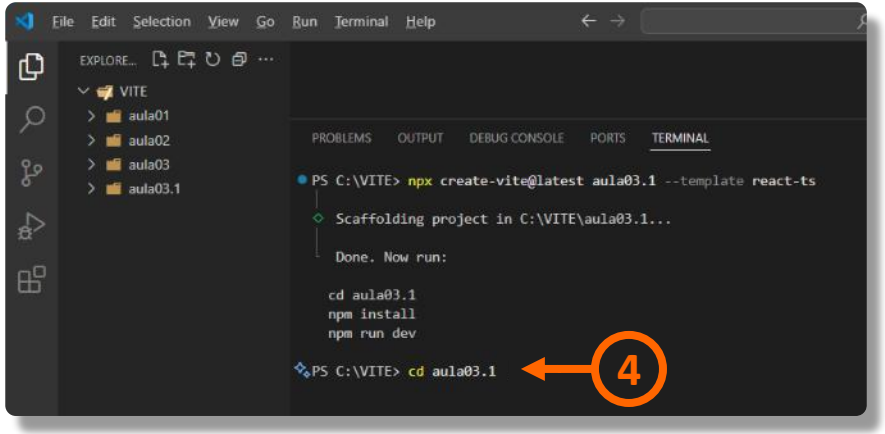


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    4. Como na imagem, vamos fazer o que se pede, no prompt digitamos '`cd aula03.1`'.

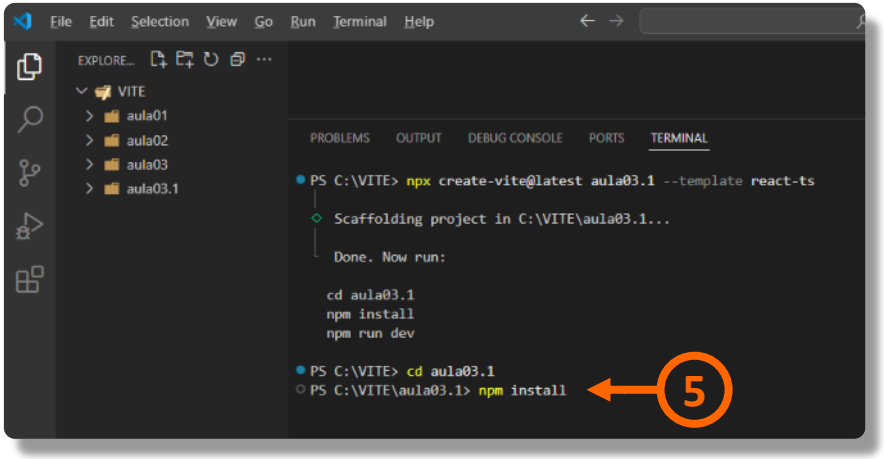


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    5. Agora, no prompt digitamos ‘`npm install`’.



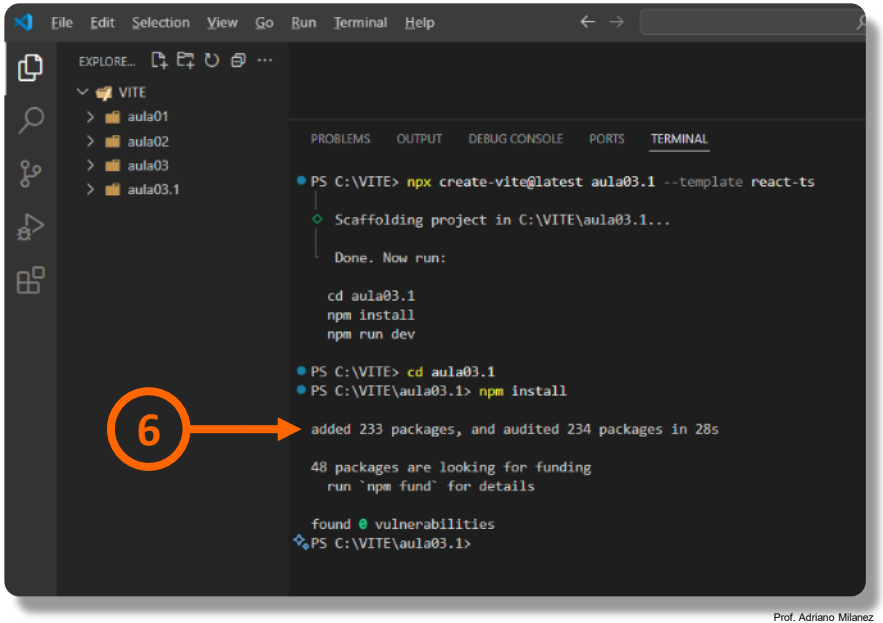
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code

6. Como resultado temos a instalação dos pacotes necessários para que nossa aplicação 'rode'.

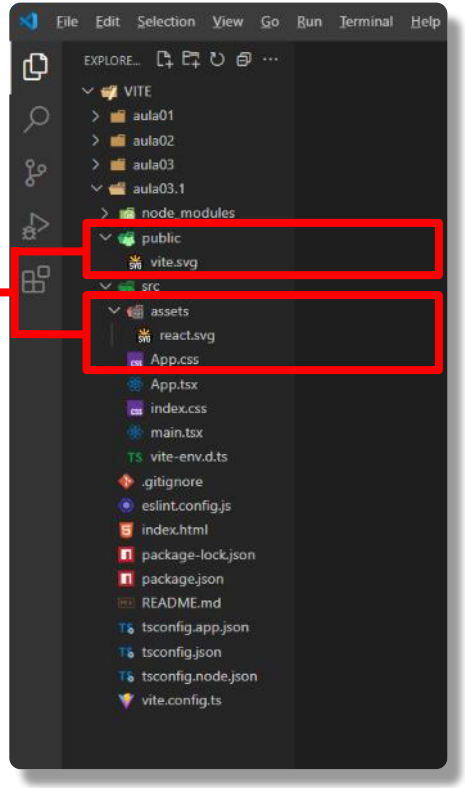


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos modificar os arquivos criados automaticamente.
  - ✓ Vamos apagar arquivos que não usaremos.
    - ✓ Conteúdo da pasta **PUBLIC**
      - ✓ VITE.SVG
    - ✓ Conteúdo da pasta **ASSETS**
      - ✓ REACT.SVG
    - ✓ Arquivos da pasta **SRC**
      - ✓ APP.CSS



Lembrando que vamos reescrever nossos arquivos MAIN.TSX e modificar o arquivo APP.TSX

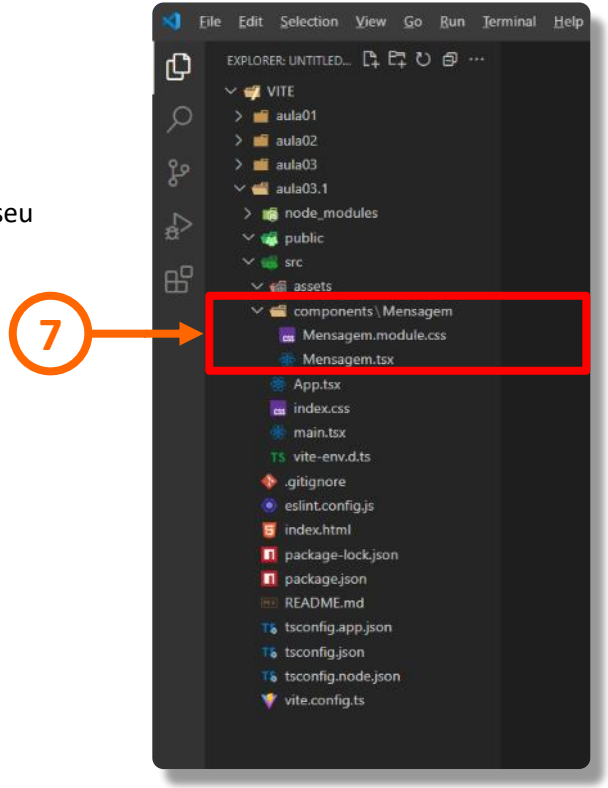
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**

7. Vamos criar o componente '**`Mensagem.tsx`**' e seu respectivo arquivo de formatação '**`Mensagem.module.css`**'.



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**

8. Vamos criar o componente ‘`SRC/Components/Mensagem/`**`Mensagem.module.css`**’

8 →

```
1. .paragrafo {
2.   color: blue;
3.   font-size: 20px;
4.   font-weight: bold;
5. }
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**

9. Vamos criar o componente ‘`SRC/Components/Mensagem/`**`Mensagem.tsx`**’

```
1. import styles from './Mensagem.module.css';
2. function Mensagem() {
3.   return(
4.     <>
5.       <p className={styles.paragrafo}>Olá, isso é um componente
         com CSS Modules!</p>;
6.     </>
7.   )
8. }
9. export default Mensagem;
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**

10. Vamos criar o componente ‘`SRC/Components/Mensagem/`**`Mensagem.tsx`**’

```
1. import styles from './Mensagem.module.css';
2. function Mensagem() {
3.   return(
4.     <>
5.       <p className={styles.paragrafo}>Olá, isso é um componente
         com CSS Modules!</p>;
6.     </>
7.   )
8. }
9. export default Mensagem;
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**

11. Vamos modificar o componente ‘SRC/**App.tsx**’

11 →

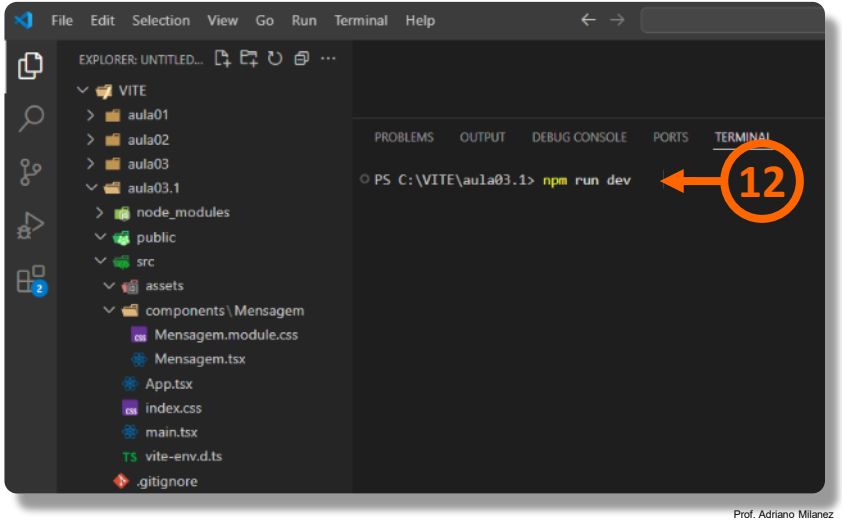
```
1. import Mensagem from "./components/Mensagem/Mensagem"
2. function App() {
3.   return (
4.     <>
5.       <Mensagem />
6.     </>
7.   )
8. }
9. export default App
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    - 12. Vamos digitar no prompt ‘`npm run dev`’ para que consigamos ‘rodar’ nossa aplicação.



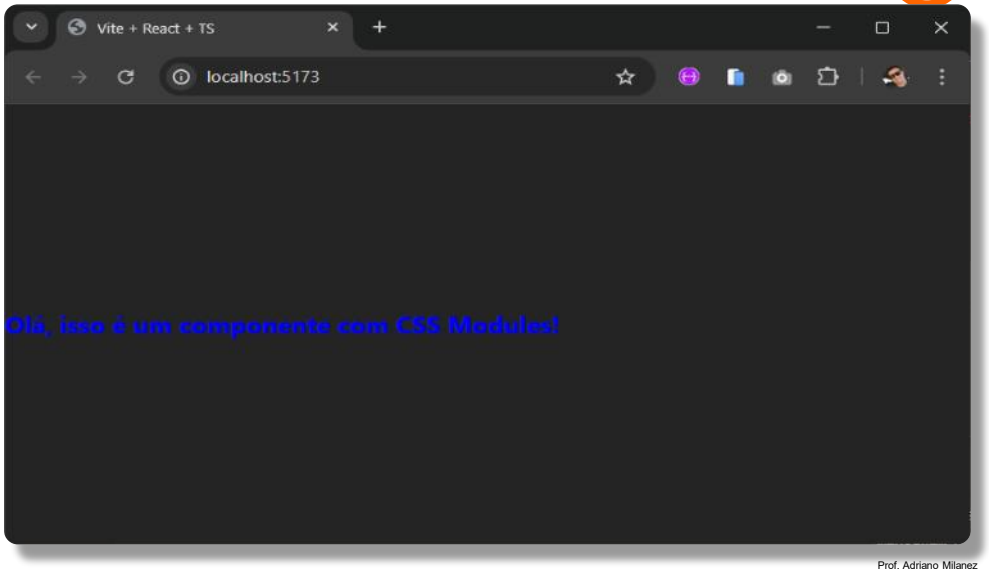
FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

2. CSS Modules

- ✓ **Vamos criar nossa QUARTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    - 13. Como resultado temos:

13



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ O **Tailwind CSS** é um framework de utilitários para estilização de interfaces. Em vez de escrever classes personalizadas no seu CSS, você usa classes prontas direto no HTML/JSX para aplicar os estilos.

- ✓ Permite que você crie layouts rápidos e responsivos aplicando classes diretamente no HTML ou JSX, sem precisar escrever CSS personalizado para cada elemento. Em vez de usar nomes de classes semânticas como em frameworks tradicionais (ex.: button, card, header), você usa classes de utilitários que aplicam propriedades CSS específicas.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Principais características**
  - ✓ **Utilitário first**
    - ✓ Em vez de criar regras CSS para um botão, você adiciona classes diretamente ao HTML/JSX. Exemplo: Classes como bg-blue-500, text-white, py-2, px-4, e rounded aplicam estilos específicos.
  - ✓ **Altamente Personalizável**
    - ✓ Você pode customizar o Tailwind através de um arquivo de configuração (tailwind.config.js). Isso permite:
      - ✓ Definir cores personalizadas
      - ✓ Adicionar espaçamentos personalizados
      - ✓ Configurar breakpoints para responsividade

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  3. Tailwind CSS

  - ✓ **Principais características**
    - ✓ **Responsividade Incorporada**
      - ✓ Tailwind tem classes de responsividade integradas. Você pode facilmente criar designs que se adaptam a diferentes tamanhos de tela. Basta adicionar prefixos como sm:, md:, lg:, etc.
    - ✓ **Componentes e Plugins**
      - ✓ Embora o Tailwind seja um framework de utilitários, você pode combinar as classes em componentes reutilizáveis. Além disso, existem plugins para facilitar a criação de animações, gradientes, tipografia, etc.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vantagens**
  - ✓ **Produtividade rápida**: Você não precisa escrever um único arquivo CSS, pois as classes utilitárias já estão prontas para usar.
  - ✓ **Customização flexível**: Embora seja baseado em utilitários, você pode personalizar e configurar de acordo com o seu projeto.
  - ✓ **Responsividade nativa**: A criação de layouts responsivos se torna muito simples com as classes de responsividade.
  - ✓ **Facilidade de manutenção**: Você aplica estilos diretamente nos elementos, sem se preocupar com folhas de estilo externas ou conflito de CSS.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Desvantagens**
  - ✓ **Classes longas:** O código HTML/JSX pode ficar com muitas classes, o que pode tornar o código visualmente mais poluído, especialmente em elementos complexos.
  - ✓ **Curva de aprendizado**: Como o Tailwind usa muitas classes utilitárias, pode ser difícil para iniciantes se acostumarem com todas as opções disponíveis.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **USE o TAILWIND se você:**
  - ✓ Precisa de produtividade rápida e não quer escrever CSS personalizado.
  - ✓ Está construindo um design responsivo e quer uma solução simples.
  - ✓ Gosta de personalização e flexibilidade.

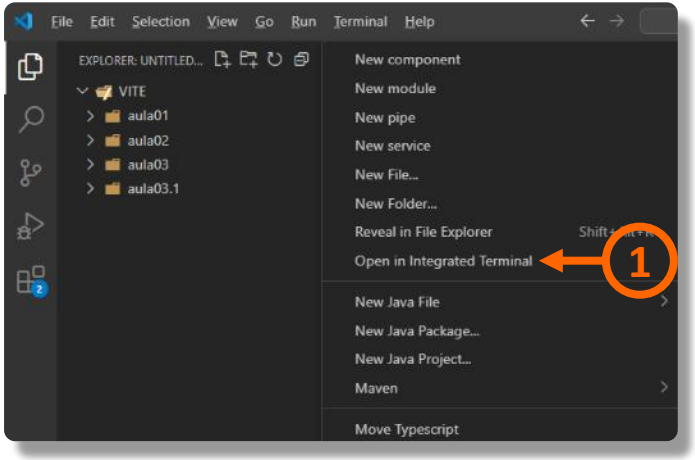
- ✓ **EVITE o TAILWIND se você:**
  - ✓ Prefere usar CSS tradicional com classes semânticas.
  - ✓ Está criando um projeto onde o número de classes utilitárias no HTML se torna muito difícil de gerenciar.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa QUINTA aplicação VITE**
  - ✓ Vamos abrir o VS Code
    1. Em uma área qualquer sem conteúdo, clicamos com o botão direito do mouse e selecionamos ‘Open in Integrated Terminal’



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa QUINTA aplicação VITE**
  - ✓ Vamos abrir o VS Code

2. Na sequência digitamos a instrução
   ‘`npx create-vite@latest nome-projeto`’
3. Acesse a pasta do projeto recém-criado ‘cd npme-projeto’
4. Digite a instrução ‘npm install tailwindcss @tailwindcss/vite’
5. Edite o arquivo ‘vite.config.ts’ e adicione as linhas em negrito

```
import { defineConfig } from 'vite’
import tailwindcss from '@tailwindcss/vite’
export default defineConfig({ plugins: [
    tailwindcss(),
],})
```

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa QUINTA aplicação VITE**
  - ✓ Vamos abrir o VS Code

    6. No seu arquivo CSS, adicione ‘@import "tailwindcss“ ’;
    7. Modifique o arquivo APP.TSX e acrescente

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

✓ **Principais abordagens de estilização**

3. Tailwind CSS

```
1.  import './App.css'
2.  function App() {
3.     return (
4.        <>
5.           <div className="min-h-screen bg-gray-100 flex items-center justify-center">
6.              <div className="bg-white p-8 rounded-xl shadow-lg text-center">
7.                 <h1 className="text-3xl font-bold text-blue-600 mb-4">Olá, Tailwind! </h1>
8.                 <p className="text-gray-600 mb-6">Esta é uma página simples usando Tailwind CSS.</p>
9.                 <button className="bg-blue-600 text-white px-6 py-2 rounded hover:bg-blue-700
                   transition">Clique aqui</button>
10.             </div>
11.          </div>
12.       </>
13.    )
14. }
15. export default App
```

11

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

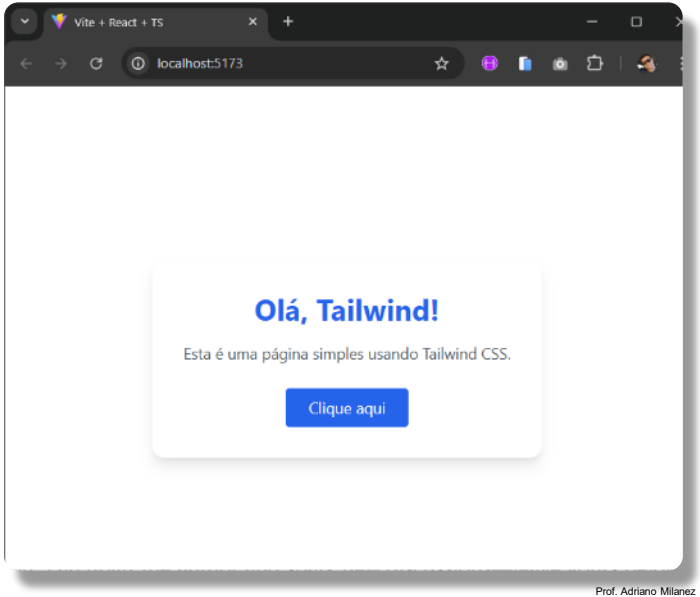
- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa QUINTA aplicação VITE**

8. Execute ‘npm run dev’ :

12

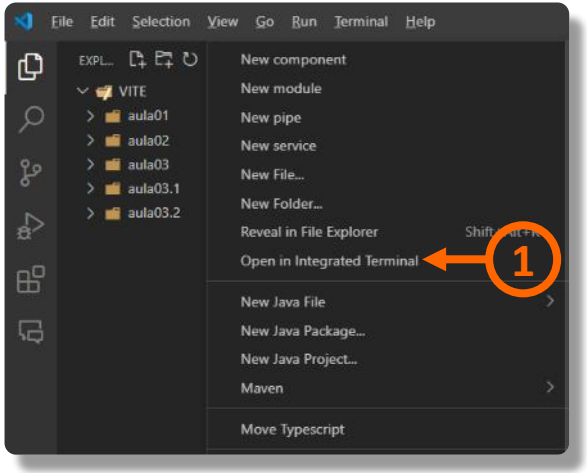


FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa SEXTA aplicação utilizando mais uma vez o TAILWIND, pois é ele que vamos utilizar como base CSS para nossas futuras aplicações.**
  - ✓ Vamos abrir o VS Code
    1. Em uma área qualquer sem conteúdo, clicamos com o botão direito do mouse e selecionamos 'Open in Integrated Terminal'



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  3. Tailwind CSS

  - ✓ **Vamos criar nossa SEXTA aplicação utilizando mais uma vez o TAILWIND, pois é ele que vamos utilizar como base CSS para nossas futuras aplicações.**
    - ✓ Vamos abrir o VS Code

      2. VAMOS REPETIR TODOS OS PASSOS DO PROJETO ANTERIOR, MODIFICANDO O NOMEDO PROJETO

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

  3. Tailwind CSS

  - ✓ **Vamos criar nossa SEXTA aplicação VITE**

    3. Vamos editar o arquivo '**app.tsx**':

```
1. function App() {
2. return (
3.   <>
4.     <div className="min-h-screen flex flex-col">
5.       {/* Cabeçalho */}
6.       <header className="bg-blue-500 text-white p-6">
7.         <h1 className="text-4xl font-bold mb-2">Bem-vindo ao Tailwind CSS!</h1>
8.         <p className="text-lg">Aprenda como criar designs incríveis com facilidade e rapidez.</p>
9.       </header>
10.      {/* Conteúdo principal cresce para ocupar o espaço disponível */}
11.      <main className="flex-1 container mx-auto p-6">
```

3

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa SEXTA aplicação VITE**

3. Vamos editar o arquivo '**app.tsx**':

```
12.         <div className="grid grid-cols-3 gap-6">
13.           <div className="bg-gray-100 p-4 rounded">
14.             <h2 className="text-2xl font-semibold mb-2">Coluna 1</h2>
15.             <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
16.           </div>
17.           <div className="bg-gray-100 p-4 rounded">
18.             <h2 className="text-2xl font-semibold mb-2">Coluna 2</h2>
19.             <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
20.           </div>
21.           <div className="bg-gray-100 p-4 rounded">
22.             <h2 className="text-2xl font-semibold mb-2">Coluna 3</h2>
```

3

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa SEXTA aplicação VITE**

3. Vamos editar o arquivo '**app.tsx**':

```
23.                <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio.</p>
24.              </div>
25.            </div>
26.              <a href="#" className="bg-green-500 text-white px-4 py-2 rounded mt-6 inline-
                 block">Clique aqui</a>
27.          </main>
28.          {/* Rodapé fixo na parte inferior */}
29.          <footer className="bg-gray-800 text-white text-center p-6">
30.            <p className="text-lg">&copy; 2024 Todos os direitos reservados.</p>
31.            <nav className="mt-4">
32.              <a href="#" className="text-blue-400 hover:text-blue-300 mx-2">Home</a>
33.              <a href="#" className="text-blue-400 hover:text-blue-300 mx-2">Sobre</a>
```

3

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa SEXTA aplicação VITE**

3. Vamos editar o arquivo '**app.tsx**':

```
34.             <a href="#" className="text-blue-400 hover:text-blue-300 mx-2">Contato</a>
35.           </nav>
36.         </footer>
37.       </div>
38.     </>
39.     )
40. }
41. export default App
```

3

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

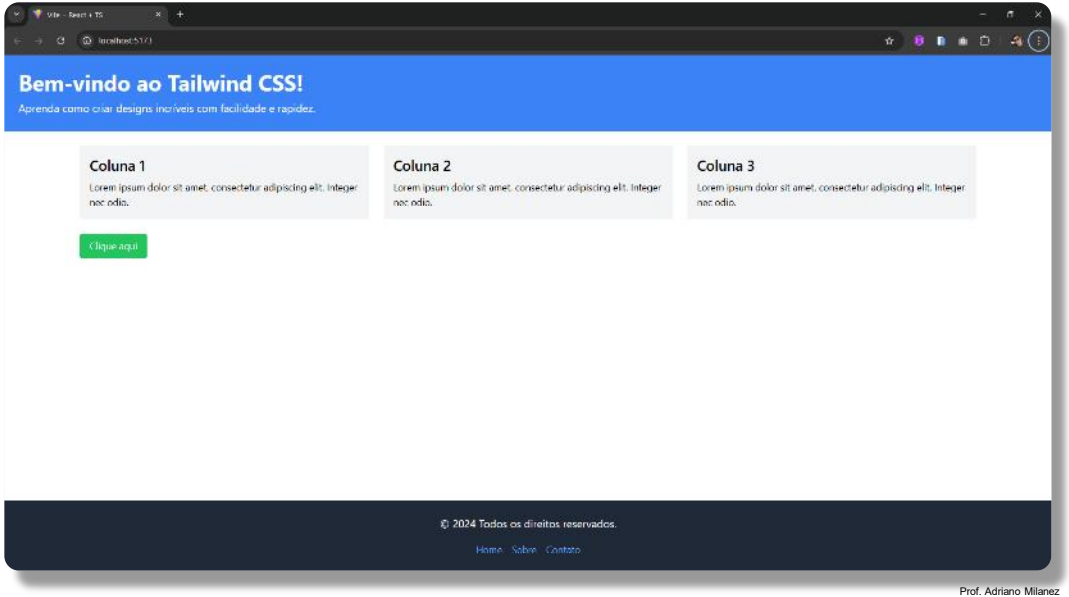
- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ **Vamos criar nossa SEXTA aplicação VITE**

4. Como resultado temos:

4



FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ Para consultar as classes do Tailwind CSS de forma simples e prática, recomendo o site oficial e algumas ferramentas que facilitam bastante:

O melhor conteúdo

- ✓ **Material Tailwind –** https://www.material-tailwind.com/docs/v3/html/button
  - ✓ Componentes prontos (botões, cards, tabelas, formulários, etc) com estilo Material Design.
  - ✓ Fácil integração com projetos Tailwind já existentes.
  - ✓ Disponível em versão HTML estático e também para React, Vue, Angular, etc.
  - ✓ Documentação clara com exemplos práticos para usar os componentes.

- ✓ **Site oficial do Tailwind CSS –** https://tailwindcss.com/docs
  - ✓ A documentação oficial, completa e organizada por categorias.
  - ✓ Muito boa para referência detalhada.

FUNDAMENTOS E BOAS PRÁTICAS DE ESTILIZAÇÃO EM REACT

- ✓ **Principais abordagens de estilização**

3. Tailwind CSS

- ✓ Para consultar as classes do Tailwind CSS de forma simples e prática, recomendo o site oficial e algumas ferramentas que facilitam bastante:

  - ✓ **Tailwind Cheat Sheet –** https://nerdcave.com/tailwind-cheat-sheet
    - ✓ Um "mapa visual" das classes mais usadas, simples e direta.
    - ✓ Ótima para achar rapidamente classes comuns.

  - ✓ **Tailwind Components –** https://tailwindcomponents.com/
    - ✓ Biblioteca com componentes prontos usando Tailwind, serve também para ver classes usadas em exemplos reais.

  - ✓ **Tailwind CSS IntelliSense (VSCode) – EXTENSÃO DO VSCODE**
    - ✓ Se usa o VSCode, instale essa extensão para autocompletar e dicas instantâneas das classes
    - ✓ Tailwind direto no editor.

Referências Bibliográficas

ABREU, Luis. Typescript - O Javascript moderno para criação de aplicações. Editora FCA, 2017.

ADRIANO, T. S. Guia prático de TypeScript. São Paulo: Casa do Código, 2021.

ANTONIO, C. Pro React: Build Complex Front-End Applications in a Composable Way With React. Apress, 2015.

BOSWELL, D; FOUCHER, T. The Art of Readable Code: Simple and Practical Techniques for Writing Better Code. Estados Unidos: O'Reilly Media, 2012.

BRITO, Robin Cris. Android Com Android Studio - Passo A Passo. Editora Ciência Moderna.

BUNA, S. React Succinctly. Estados Unidos: [s.n], 2016. Disponível em: <www.syncfusion.com/ebooks/reactjs_succinctly>. Acesso em: 12 de janeiro de 2025.

CHEHADE, Adib; CHEHADE, Nadia. Fullstack React with TypeScript and Next.js: Build scalable and production-ready full stack web apps. 1. ed. Birmingham: Packt Publishing, 2023.

CHIN, John. Next.js Quick Start Guide: Server-side rendering done right. Birmingham: Packt Publishing, 2019.

FACEBOOK. React: Getting Started. React Docs, 2019a. Disponível em: <reactjs.org/docs/react-api.html>. Acesso em: 13 de julho de 2025.

_____. React Without ES6. React Docs, 2019b. Disponível em: <reactjs.org/docs/react-without-es6.html>. Acesso em: 10 de julho de 2025.

_____. React Without JSX. React Docs, 2019c. Disponível em: <reactjs.org/docs/react-without-jsx.html>. Acesso em: 10 de julho de 2025.

FREEMAN, Eric ROBSON, Elisabeth. Use a Cabeça! Programação em HTML5. Rio de Janeiro: Editora Alta Books, 2014

GACKENHEIMER, C. Introduction to React: Using React to Build scalable and efficient user interfaces.[s.i.]: Apress, 2015.

GOLDBERG, Josh. Aprendendo TypeScript: Melhore Suas Habilidades de Desenvolvimento web Usando JavaScript Type-Safe. São Paulo: Novatec. 2022

HUDSON, P. Hacking with React. 2016. Disponível em: <www.hackingwithreact.com/read/1/3/introduction-to-jsx>. Acesso em: 13 julho de 2025.

KOSTRZEWA, D. Is React.js the Best JavaScript Framework in 2018? 2018. Disponível em: <hackernoon.com/is-react-js-the-best-JavaScript-framework-in-2018-264a0eb373c8>. Acesso em: 25 julho de 2025.

MARTIN, R. Clean Code: A Handbook of Agile Software Craftsmanship. Estados Unidos: Prentice Hall, 2009.

MDN WEB DOCS. Guia JavaScript. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>. Acessado em 25 de julho de 2025.

Referências Bibliográficas

NELSON, J. Learn React's Fundamentals Without the Buzzwords? 2018. Disponível em: <jamesknelson.com/learn-react-fundamentals-sans-buzzwords>. Acesso em: 12 julho de 2025.

NEXT.JS. *Tailwind CSS com Next.js – Guia Oficial*. Disponível em: https://tailwindcss.com/docs/guides/nextjs. Acesso em: 10 de julho de 2025.

_____. The React Framework for the Web. Disponível em: https://nextjs.org/. Acesso em: 23 de julho de 2025.

NIELSEN, J. Response Times: The 3 Important Limits. 1993. Disponível em: <www.nngroup.com/articles/response-times-3-important-limits>. Acesso em: 10 julho de 2025.

O'REILLY, T. What is Web 2.0: Design Patterns and Business Models for the Next Generation of Software. 2005. Disponível em: <www.oreilly.com/pub/a/web2/archive/what-is-web-20.html#mememap>. Acesso em: 10 de julho de 2025.

PANDIT, N. What Is ReactJS and Why Should We Use It? 2018. Disponível em: <www.c-sharpcorner.com/article/what-and-why-reactjs>. Acesso em: 12 de julho de 2025.

RAUSCHMAYER, A. Speaking JavaScript: An In-Depth Guide for Programmers. Estados Unidos: O'Reilly Media, 2014.

REACTIVA. O arquivo package-lock.json. Disponível em: <https://nodejs.reativa.dev/0020-package-lock-json/index>. Acessado em 13 de julho de 2025.

_____. O guia do package.json. Disponível em: <https://nodejs.reativa.dev/0019-package-json/index>. Acessado em 13 de julho de 2025.

RICOY, L. Desmitificando React: Uma Reflexão para Iniciantes. 2018. Disponível em: <medium.com/trainingcenter/desmitificando-react-uma-reflex%C3%A3o-para-iniciantes-a57af90b6114>. Acesso em: 13 julho de 2025.

SHARMA, Nabendu Biswas; UDAYAN, Kuldeep. Building Microfrontends with React 18 and Next.js 13: A complete guide to building modern microfrontend-based web apps using React and Next.js. 1. ed. Birmingham: Packt Publishing, 2023.

SILVA, Maurício Samy. Ajax com jQuery: requisições Ajax com a simplicidade de jQuery. São Paulo: Novatec Editora, 2009.

_____. Construindo Sites com CSS e XHTML. Sites Controlados por Folhas de Estilo em Cascata. São Paulo: Novatec, 2010.

_____. CSS3 - Desenvolva aplicações web profissionais com o uso dos poderosos recursos de estilização das CSS. São Paulo: Novatec Editora, 2010.

STACKOVERFLOW. Most Popular Technologies: Web Frameworks. Developer Survey Results, StackOverflow, 2019. Disponível em: <insights.stackoverflow.com/survey/2019#technology>. Acesso em: 13 de julho de 2025.

SWC. Rust-based platform for the Web. 2024 Disponível em <https://swc.rs/>. Acessado em 23 de julho de 2025.

Referências Bibliográficas

TAILWIND CSS. *Tailwind CSS Documentation*. Disponível em: https://tailwindcss.com/docs. Acesso em: 10 de julho de 2025.

TAILWIND LABS. *Curso gratuito de Tailwind CSS*. Disponível em: https://play.tailwindcss.com. Acesso em: 10 de julho de. 2025.

STACK OVERFLOW. *Sticky Footer with Tailwind CSS*. Disponível em: https://stackoverflow.com/questions/43556907/sticky-footer-with-tailwind-css. Acesso em: 10 de julho de 2025.

TAILWIND CSS. *Making a Full Height Layout*. Disponível em: https://tailwindcss.com/docs/min-height#making-a-full-height-layout. Acesso em: 10 de fevereiro de 2025.

VERCEL. *Criando layouts responsivos no Next.js*. Disponível em: https://nextjs.org/docs/basic-features/layouts. Acesso em: 10 de julho de 2025.

VERCEL, Inc. Next.js: The React Framework. Disponível em: https://nextjs.org/. Acesso em: 25 de julho de 2025.

VERCEL, Inc. Next.js Documentation. Disponível em: https://nextjs.org/docs. Acesso em: 25 de julho de 2025.

W3C. HTML5 - A linguagem de marcação que revolucionou a web. São Paulo: Novatec Editora, 2010.

_____. A vocabulary and associated APIs for HTML and XHTML. Disponível em <https://www.w3.org/TR/2018/SPSD-html5-20180327/>. Acessado em 22 de julho de 2025.

_____. Cascading Style Sheets, level 1. Disponível em <https://www.w3.org/TR/2018/SPSD-CSS1-20180913/>. Acessado em 22 de julho de 2025.

_____. Cascading Style Sheets, level 2 Revision 2. Disponível em <https://www.w3.org/TR/2016/WD-CSS22-20160412/>. Acessado em 22 de julho de 2025.

_____. Cascading Style Sheets, level 2. Disponível em <https://www.w3.org/TR/2008/REC-CSS2-20080411/>. Acessado em 22 de julho de 2025.

_____. Cascading Style Sheets, level 3. Disponível em <https://www.w3.org/TR/css-syntax-3/>. Acessado em 22 de julho de 2025.

_____. HTML 3.2 Reference Specification. Disponível em <https://www.w3.org/TR/2018/SPSD-html32-20180315/>. Acessado em 20 de julho de 2025.

_____. HTML 4.0 Specification. Disponível em <https://www.w3.org/TR/1998/REC-html40-19980424/>. Acessado em 20 de julho de 2025.

_____. HTML 4.01 Specification. Disponível em <https://www.w3.org/TR/2018/SPSD-html401-20180327/>. Acessado em 20 de julho de 2025.

Referências Bibliográficas

W3C.Cascading Style Sheets, level 2 Revision 1. Disponível em <https://www.w3.org/TR/CSS2/>. Acessado em 20 de julho de 2025.

WIKIPEDIA. JavaScript. Disponível em <https://pt.wikipedia.org/wiki/JavaScript>. Acessado em 20 de julho de 2025.

Dúvidas?

Críticas?

Sugestões?

Ameaças?